C PROGRAMMING — ASSIGNMENT 1

TASK 3: DEFINE THE CONSTANTS

CO1 • Bloom's Level: K3 – Apply • Marks: 3

Assignment title: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

1. Objective

Define tariff values and limits in one place so that tariff changes can be made consistently. The supplied program uses named macros for slab limits, rates in paise, and the fixed charge. The bill is calculated in integer paise to avoid floating-point rounding in the charge calculation.

2. Constants used

Macro	Value	Meaning
SLAB1_UNITS	100	First 100 units are in the free slab.
SLAB2_UNITS	200	Upper limit of the second slab.
RATE1_PAISE	0	Rate for the first slab: 0 paise per unit.
RATE2_PAISE	225	Rate for units 101–200: 225 paise (■2.25) per unit.
RATE3_PAISE	450	Rate above 200 units: 450 paise (■4.50) per unit.
FIXED_CHARGE_PAISE	5000L	Fixed charge of ■50.00, stored as paise.

3. Completed C program

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;

    if (units > SLAB1_UNITS) {
        int slab2 = units < SLAB2_UNITS
                  ? units - SLAB1_UNITS
                  : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2 * RATE2_PAISE;
    }

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

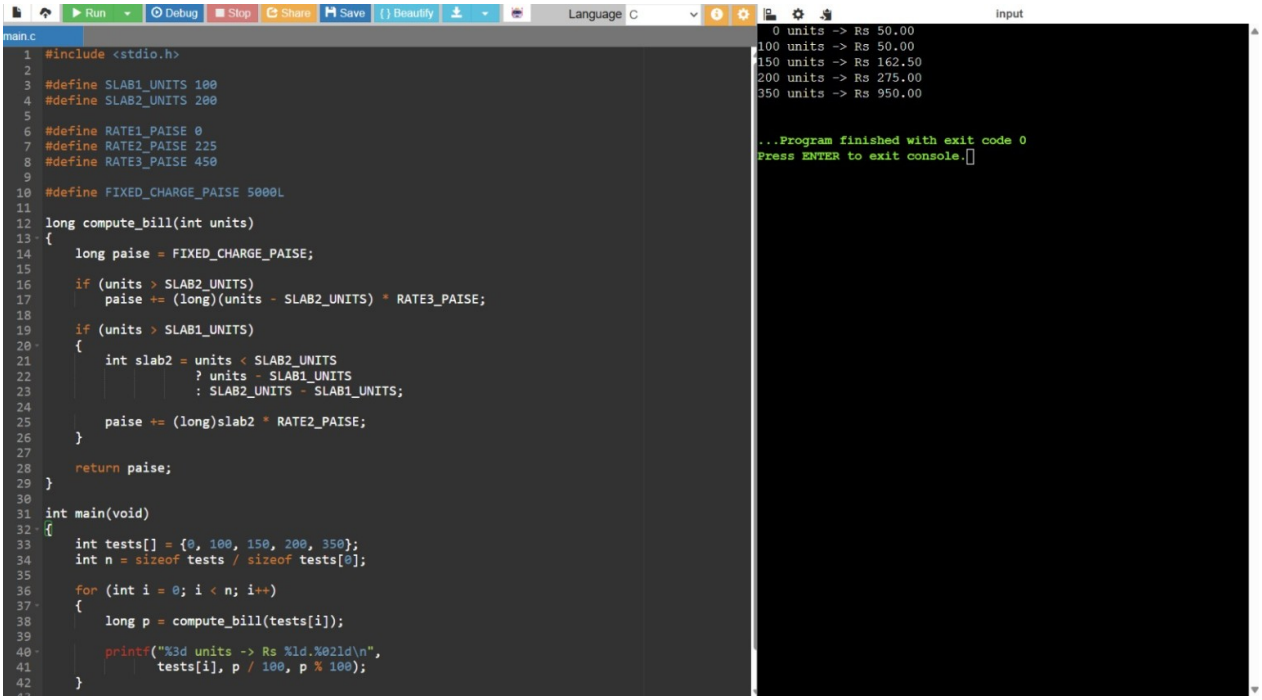
4. Test cases and observed output

Units	Energy charge	Fixed charge	Total bill
0	■0.00	■50.00	■50.00
100	■0.00	■50.00	■50.00

Units	Energy charge	Fixed charge	Total bill
150	■112.50	■50.00	■162.50
200	■225.00	■50.00	■275.00
350	■900.00	■50.00	■950.00

The output in the supplied screenshot matches these totals: 0 → Rs 50.00; 100 → Rs 50.00; 150 → Rs 162.50; 200 → Rs 275.00; and 350 → Rs 950.00.

5. Execution screenshot




```c
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;

    if (units > SLAB1_UNITS)
    {
        int slab2 = units < SLAB2_UNITS
                  ? units - SLAB1_UNITS
                  : SLAB2_UNITS - SLAB1_UNITS;

        paise += (long)slab2 * RATE2_PAISE;
    }

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);

        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
```

Figure 1. Supplied source code and successful output for the tariff-slab billing program.

6. #define, const and enum

#define: The preprocessor replaces the macro name with its value before compilation. It has no declared C type and does not create a variable by itself. It is convenient for simple compile-time tariff values used consistently throughout the program.

const: A const object has a C type and follows normal scope and type-checking rules. It is easier to inspect in a debugger, but it is an object rather than a preprocessor substitution.

enum: An enumeration is suitable for named integral choices such as consumer categories or status codes. It is not ideal for fractional currency rates, though integer paise values could be represented as enumerators when their values fit the implementation's enum range.

For this program, #define is a straightforward choice because the task specifically asks for tariff macros and a single place to revise the values. For larger programs, typed const values or a runtime configuration file may offer stronger type checking or easier tariff updates without recompilation.

7. Alternative concept and complexity

If the macros were replaced with const variables, the bill computation would remain the same, but names would be typed C objects with scope and debugger visibility. If an enum were used, only integral named constants would be appropriate. The running time is unchanged: the tariff arithmetic performs a fixed number of operations, so it is O(1) per bill. The storage difference is generally negligible; a macro itself occupies no storage, while a const object may occupy storage depending on compiler optimisation and how it is used.

8. Compile and run

```
gcc -std=c11 -Wall -Wextra task03.c -o task03
./task03
```

Conclusion

Named tariff macros keep slab limits, rates and the fixed charge centralised. The program calculates the bill in paise and converts to rupees only for output. The supplied execution confirms the expected totals for all five test cases.